

Developing an Angular Application Using Redux for State Management



This article explains the development of an Angular application, using Redux for state management, interfacing with the Express framework for data and using Angular CLI as the front-end build pipeline.

Angular is a JavaScript framework for building user interfaces and can be used to build single page applications. There is a large amount of state that needs to be managed by single page applications. The state includes server responses, cached and locally created data, and various UI states. Redux is a popular framework for managing the state. The data displayed by the client side UI can be provided from a back-end server. Express is a commonly used Web framework based on Node.js. This article explains the development of an Angular application using Redux for state management and interfacing with the Express framework for data along with using Angular CLI for setting up the development environment. Angular CLI seamlessly handles the front-end build pipeline, enabling us to focus on writing the application logic.

To get an understanding of this article, readers should have a knowledge of the basics of Angular, Redux, Express.js, and Node.js concepts.

Contacts application

The concepts in developing an application using Angular, Redux and Express will be illustrated through a *Contacts* application. We will keep the application simple, as the main objective is to illustrate the various concepts in an

easy-to-understand manner. Using the app, we will be able to view the *Contacts* list, add a new contact and delete a contact. We will also use middleware called *redux-thunk* to perform the action asynchronously.

Setting up the development environment

Let's develop the app using Windows as the development machine, as follows:

1. Install *Node.js* from <https://nodejs.org>. Download the 32-bit or 64-bit binary depending upon the architecture of the system.
2. Install *Angular CLI*, using which we will start building the *Contacts* application:

```
npm install -g @angular/cli
```

3. Generate the basic code template, which we will modify to add the features required for the *Contacts* application.

```
ng new contacts
```

This will create a directory called *contacts* in the current folder. Inside that folder, you will see an initial project structure generated, and the required dependencies are automatically installed.